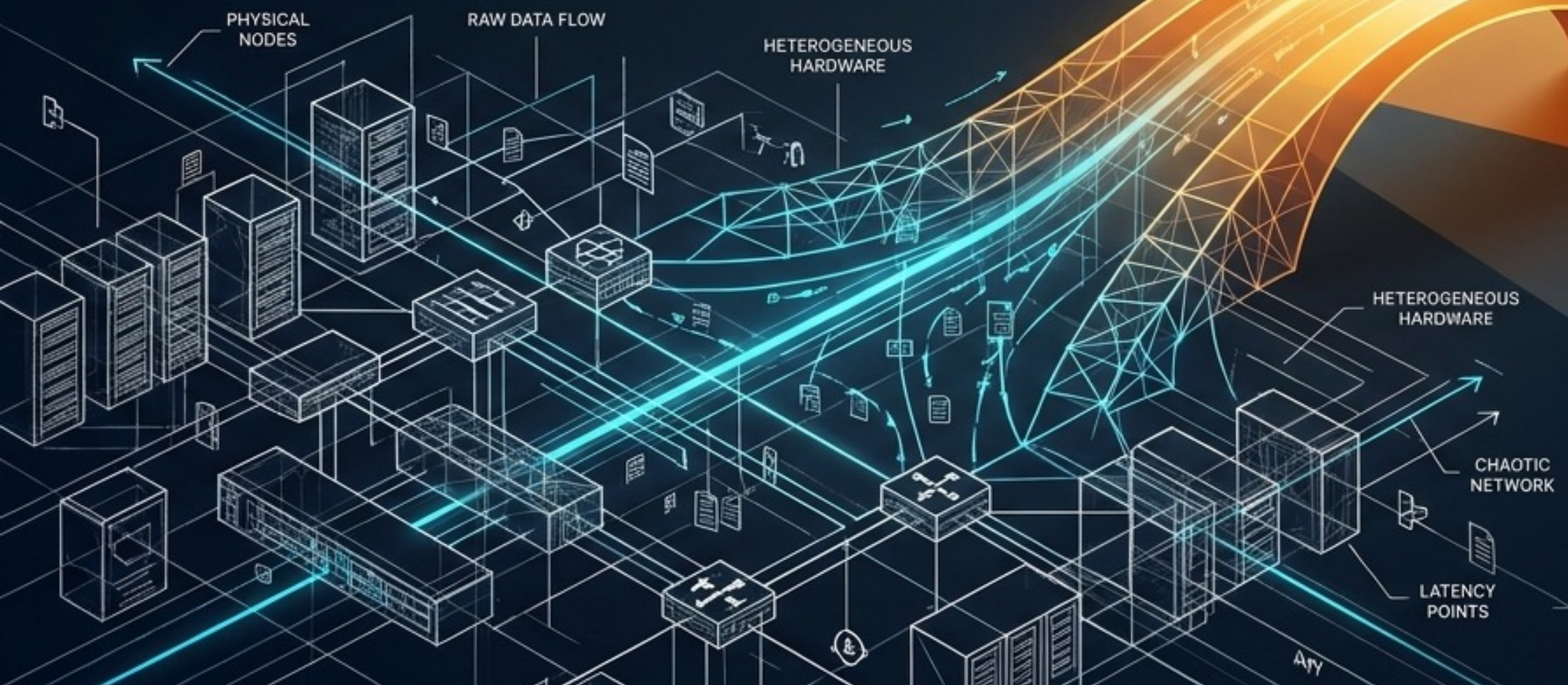


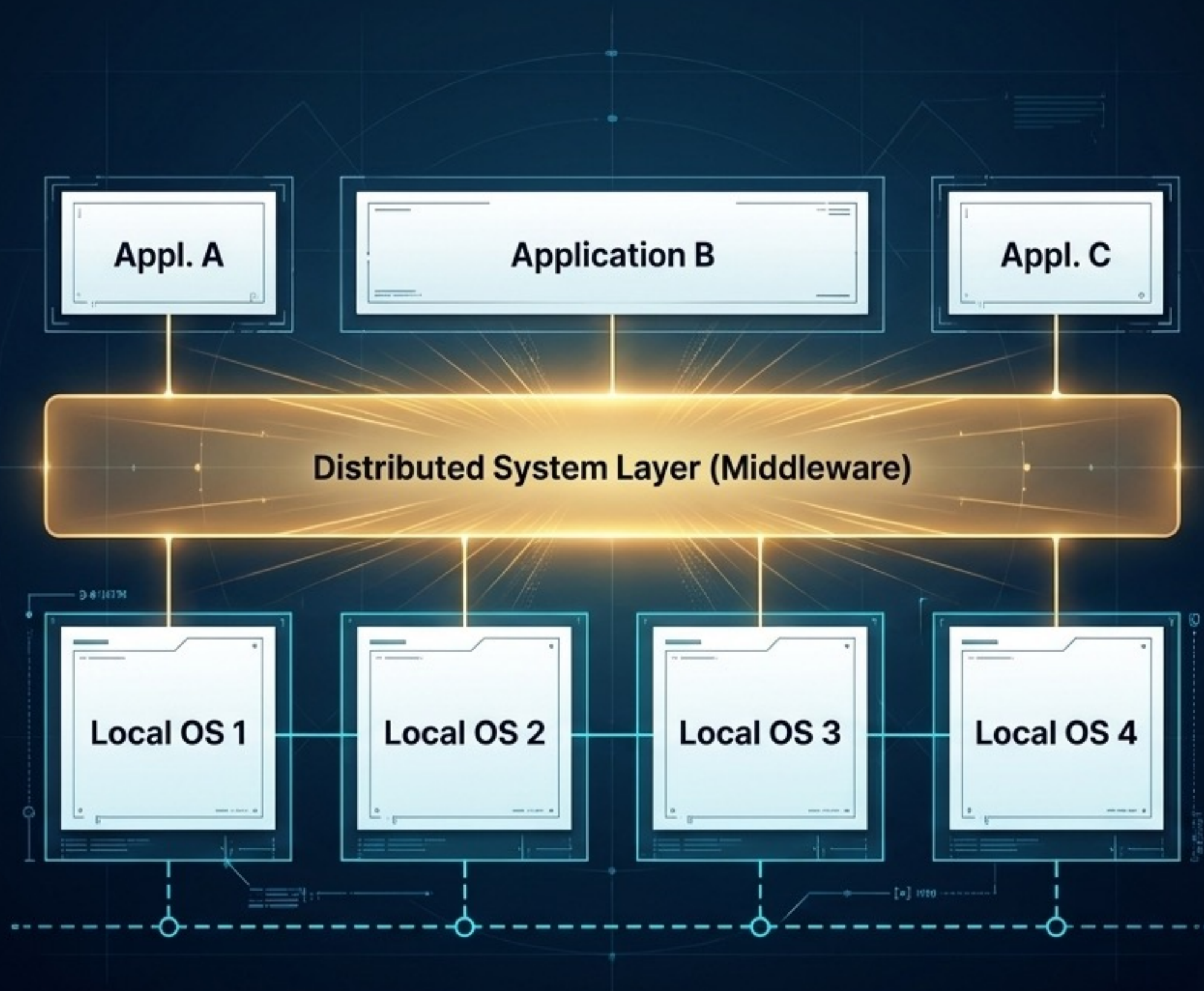
Lecture 01 | Undergraduate ICT

The Illusion of One

An Overview of Distributed Systems
Principles, Architectures, and Paradigms

Based on Tanenbaum & van Steen (Chapters 1 & 2)





Defining the Illusion

"A distributed system is a collection of independent computers that appears to its users as a single coherent system."

- **Heterogeneity & Autonomy:** Underlying components are independent and wildly diverse.
- **The Single-System View:** Users interact uniformly, regardless of interaction location or time.
- **The Middleware Layer:** The critical software bridge placed logically between applications and local operating systems to hide the seams.

The Four Rules of the Illusion

Resource Accessibility

Connecting users to remote resources for economics, sharing, and collaboration.

Distribution Transparency

Systematically hiding the physical realities of the network from the user.

Openness

Standardized rules, interoperability, and the ability to seamlessly swap components.

Scalability

The capacity to grow infinitely in size, geography, and administration without performance collapse.

Goal 1: Resource Accessibility



The Economic Driver:

Sharing expensive peripherals, high-performance storage, and specialized compute power among distributed users.

The Collaboration Engine:

Enabling virtual organizations (groupware) and global electronic commerce.

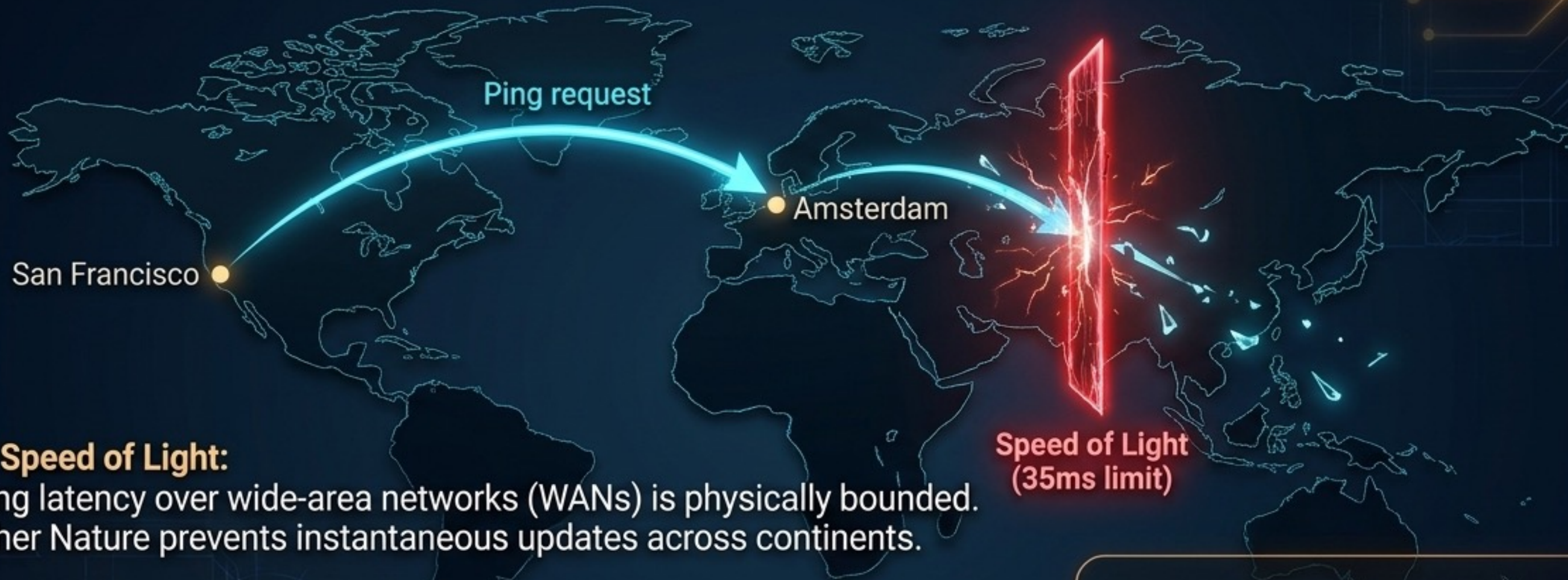
The Consequence - Security:

Increased connectivity drastically increases vulnerability. Storing data remotely necessitates severe access control, authentication, and privacy protection.

Goal 2: The Transparency Matrix

Transparency Type	What it hides	Real-world example
Access	Hides data representation & access methods	Cross-platform file manipulation.
Location	Hides where a resource is	URLs giving no physical location clues.
Migration	Hides that a resource has moved	Shifting a database to a new server offline.
Relocation	Hides that a resource is currently moving	Wireless laptop keeping connection while roaming.
Replication	Hides multiple copies	Users see one name, system manages identical replicas.
Concurrency	Hides competitive sharing	Locking mechanisms allowing simultaneous database queries.
Failure	Hides crash & recovery	Masking a dead node as a slow network.

The Physical Limits of Transparency



The Speed of Light:

Hiding latency over wide-area networks (WANs) is physically bounded. Mother Nature prevents instantaneous updates across continents.

The Performance Trade-off:

Endlessly retrying to mask a transient server failure can slow down the entire system. Sometimes it is better to expose the failure.

Context-Awareness:

In ubiquitous computing, exposing distribution is the goal (e.g., printing to a physically nearby printer rather than a corporate default).

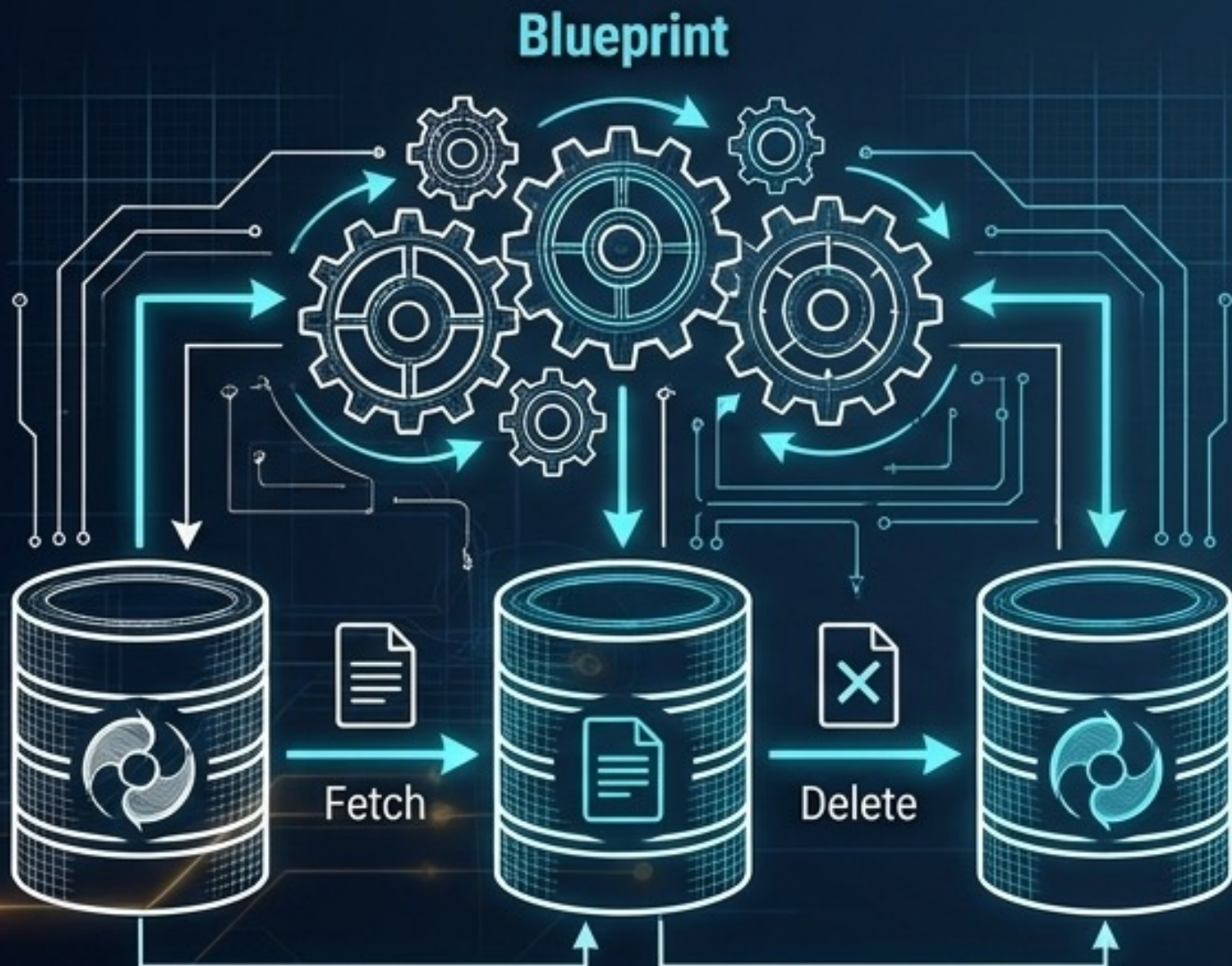
“You know you have a distributed system when the crash of a crash of a computer you’ve never heard of stops you from getting any work done.”

- Lamport’s Reality Check

Goal 3: Openness & The Separation Principle

Neutral Interfaces: Standardized protocols and complete Interface Definition Languages (IDLs) ensure interoperability and portability.
Monolithic vs. Extensible: Moving away from closed systems toward small, easily replaceable components.

Mechanism (The How): The browser provides the facility to store documents locally.



Policy (The What): The user dictates which documents are stored, for how long, and when to flush the cache.

Seamless Surface

Cache Enabled	☒	Auto-Flush	☒
Duration (Days)	10	Max File Size (MB)	30
<input type="range"/>		<input type="range"/>	
Content Rules	Flush Options		
v	v		

Crucial Principle: *Separate Policy from Mechanism*

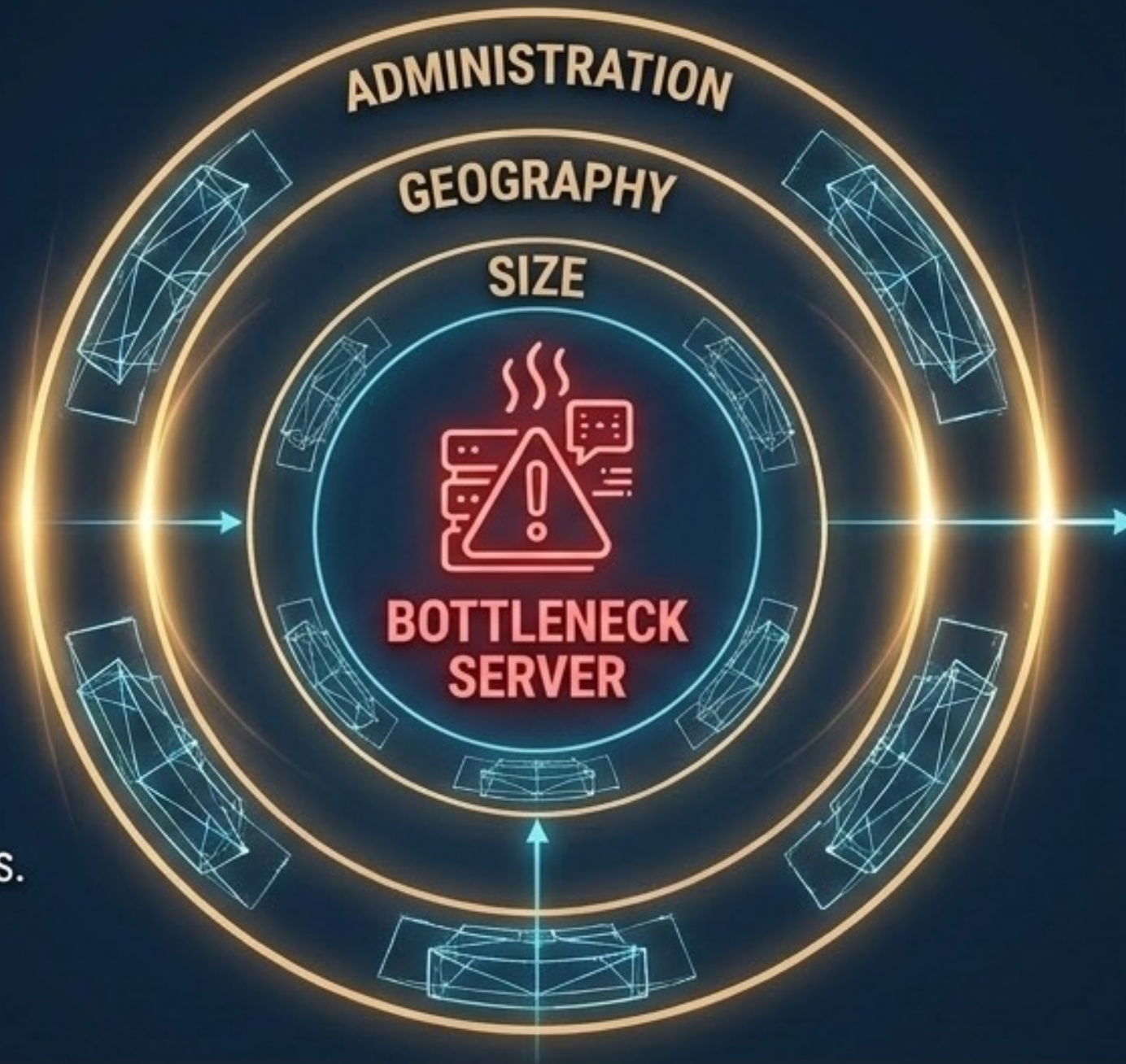
Goal 4: Scalability Dimensions & Centralization Pitfalls

Three Dimensions of Scale:

Size: Adding more users/resources seamlessly.

Geography: Masking distance between nodes.

Administrative: Managing trust and policies across independent domains.



The Pitfalls of Centralization:

Centralized Services: A single server handling all log all requests will crash.

Centralized Data: A single database limits total throughput.

Centralized Algorithms: Routing all logic through one master node creates an unscalable bottleneck.

Rules for Decentralized Algorithms:

- No machine has complete state;
- Decisions use local info;
- Failure of one doesn't ruin the algorithm;
- No implicit global clock.

When the Illusion Breaks: 8 False Assumptions

Formulated by Peter Deutsch regarding first-time distributed systems design.

1. The network is reliable.

2. The network is secure.

3. The network is homogeneous.

4. The topology does not change.

5. Latency is zero.

6. Bandwidth is infinite.

7. Transport cost is zero.

8. There is one administrator.

Hiding the Seams: Three Scaling Techniques

Technique 1: Latency Hiding



Technique 1: Latency Hiding

Shift from synchronous to asynchronous communication.

Example: Shipping code to the client (JavaScript form validation locally).

Technique 2: Distribution



Technique 2: Distribution

Partitioning data and computations across multiple machines.

Example: The Domain Name System (DNS) splitting global naming into zones.

Technique 3: Replication & Caching



Technique 3: Replication & Caching

Creating local copies to balance load and leverage geographical locality.

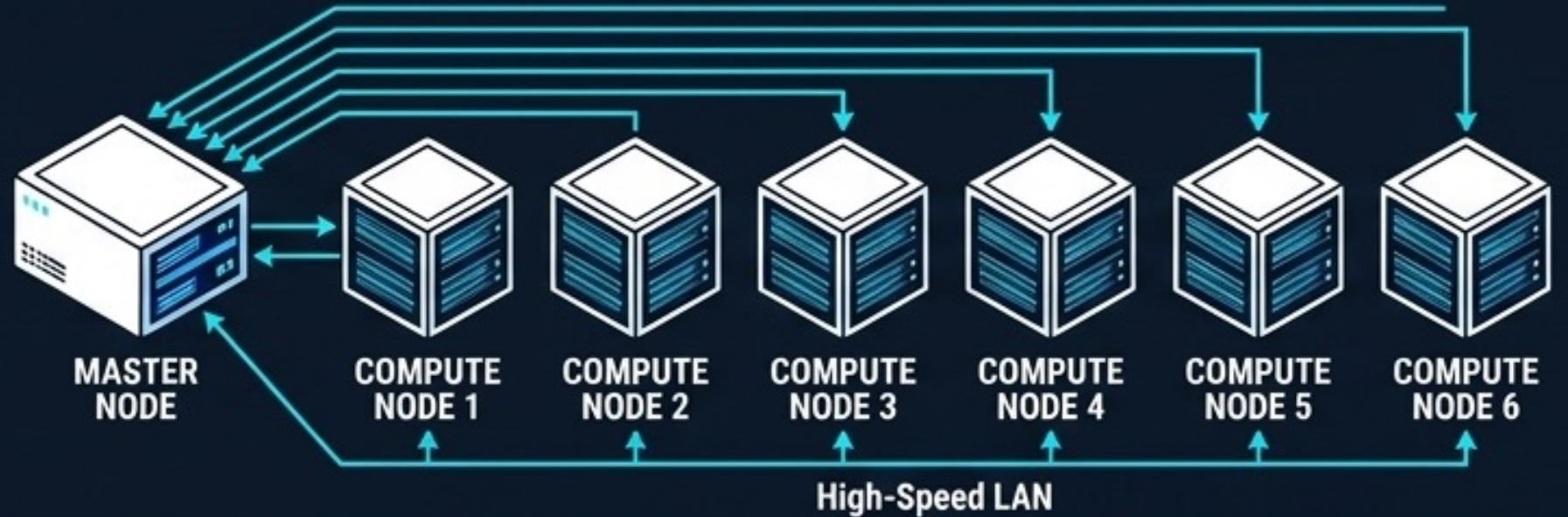
Trade-off: Introduces severe consistency problems.

System Venues I: HPC & Grid Computing

High-Performance Computing (HPC)

Cluster Computing: Homogeneous PCs tightly connected over high-speed LANs (e.g., Beowulf clusters with master/compute node split).

MOSIX: Provides symmetric process migration, moving entire operating systems between nodes.



Grid Computing

Heterogeneous systems forming virtual organizations across administrative domains.

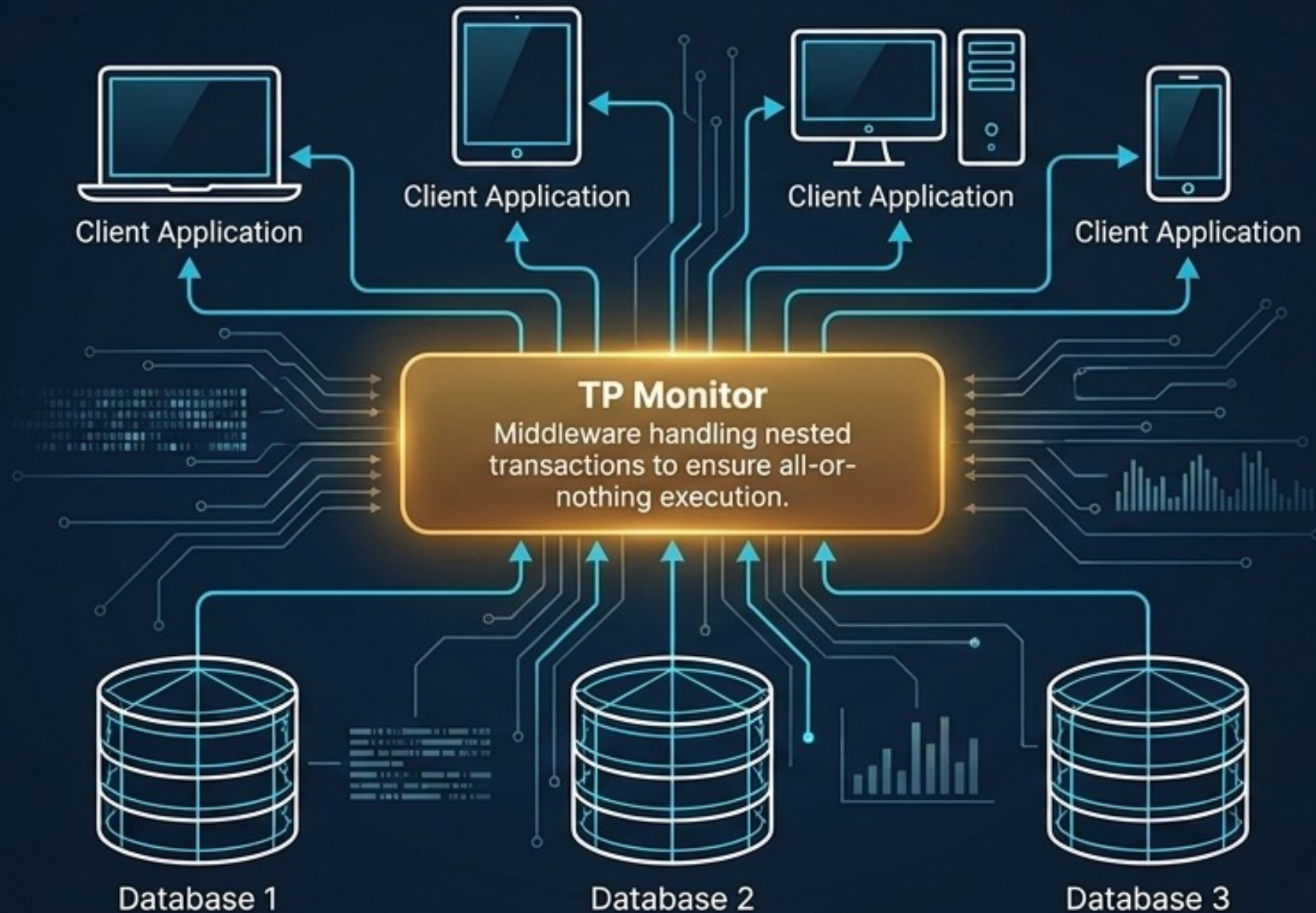
Foster's Layered Architecture: Fabric (hardware), Connectivity (secure routing), Collective (resource allocation), Application.



System Venues II: Distributed Information Systems

Transaction Processing Systems

Ensuring ACID properties across distributed databases.



Enterprise Application Integration (EAI)

- **RPC** (Remote Procedure Call): Tight coupling via remote procedure requests.
- **RMI** (Remote Method Invocation): Applying RPC to distributed objects.
- **MOM** (Message-Oriented Middleware): Decoupling via publish/subscribe messaging systems.

System Venues III: Pervasive & Embedded Systems

The Paradigm Shift

Default instability. Small, battery-powered, mobile, wireless nodes with no central human administration.

Grimm's Requirements

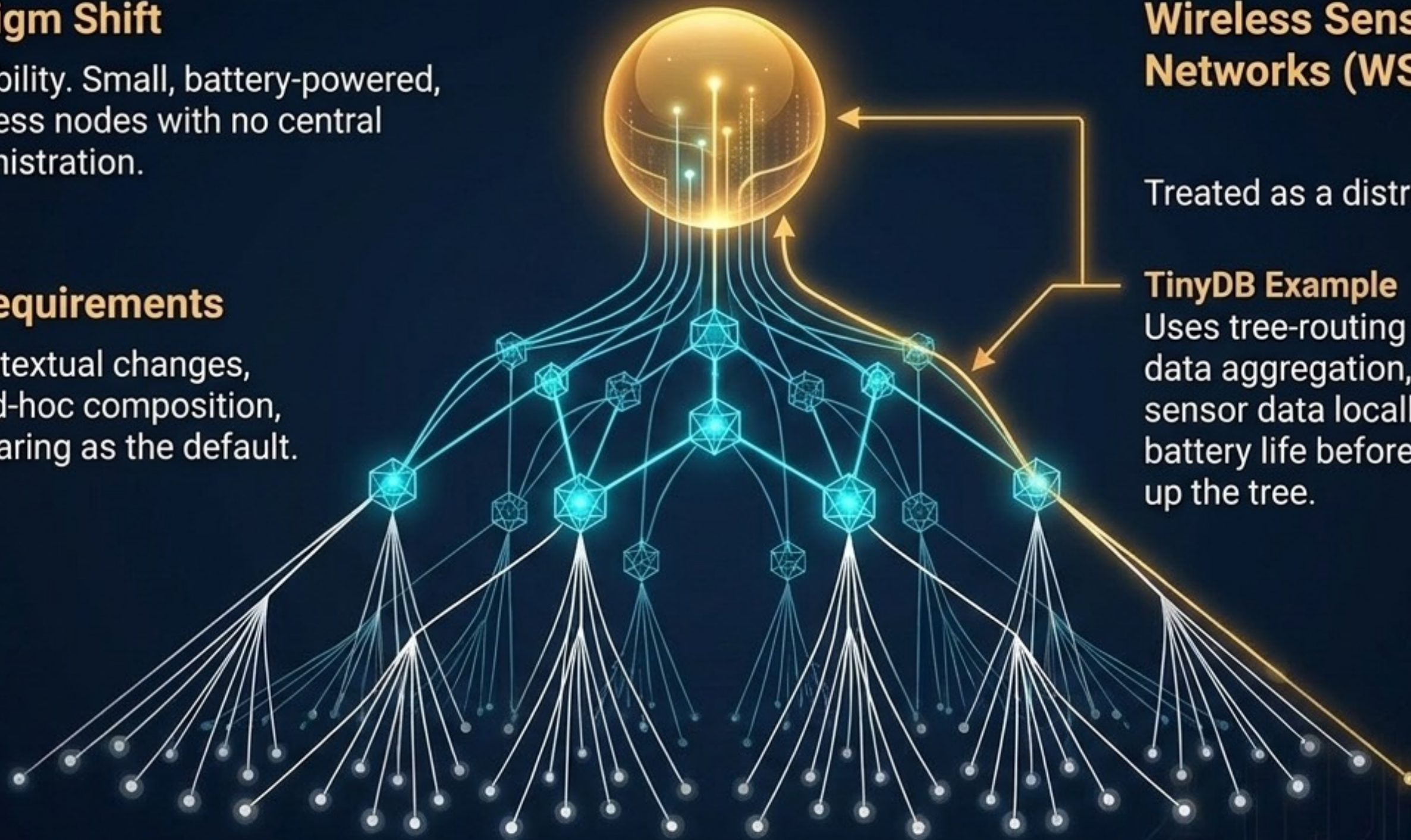
Embrace contextual changes, encourage ad-hoc composition, recognize sharing as the default.

Wireless Sensor Networks (WSNs)

Treated as a distributed database.

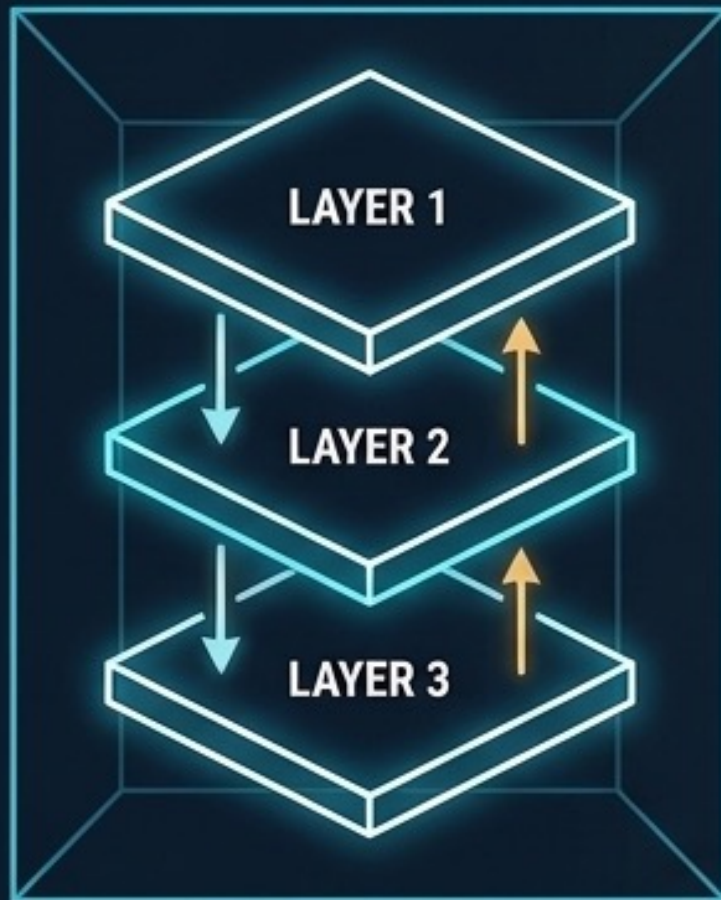
TinyDB Example

Uses tree-routing for in-network data aggregation, filtering raw sensor data locally to preserve battery life before passing it up the tree.



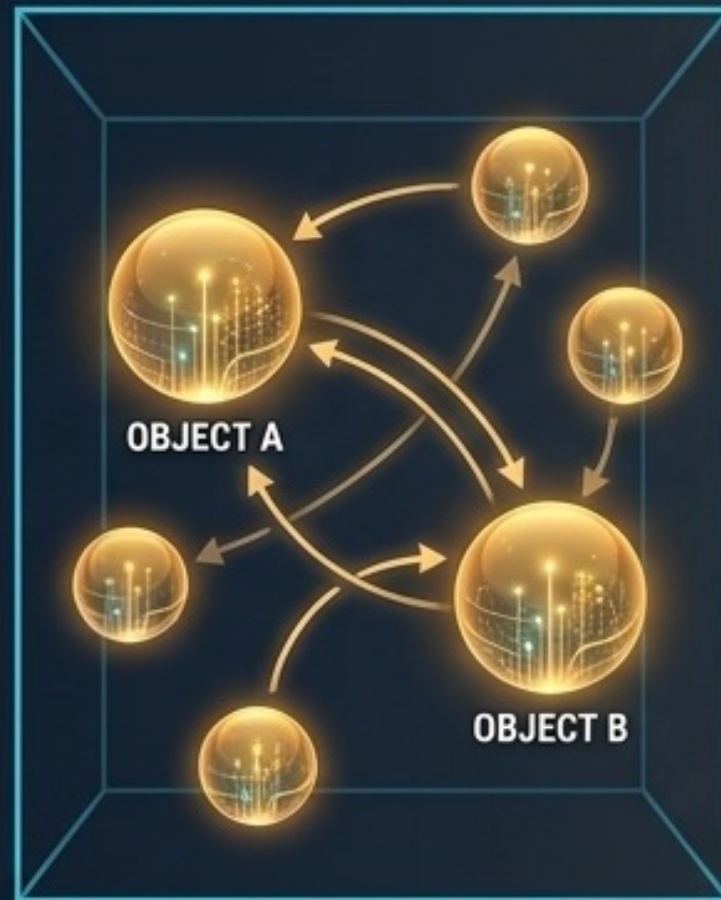
Pervasive Network Tree

Wiring the Trick: Logical Software Styles



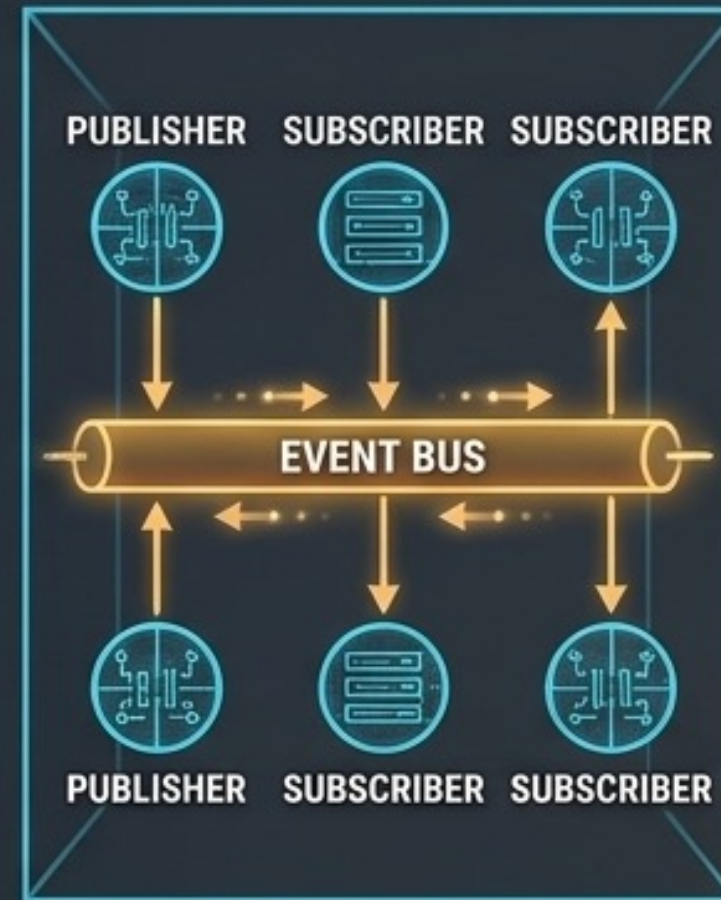
1. Layered

Downward-flowing control.
Requests move down,
responses move up.



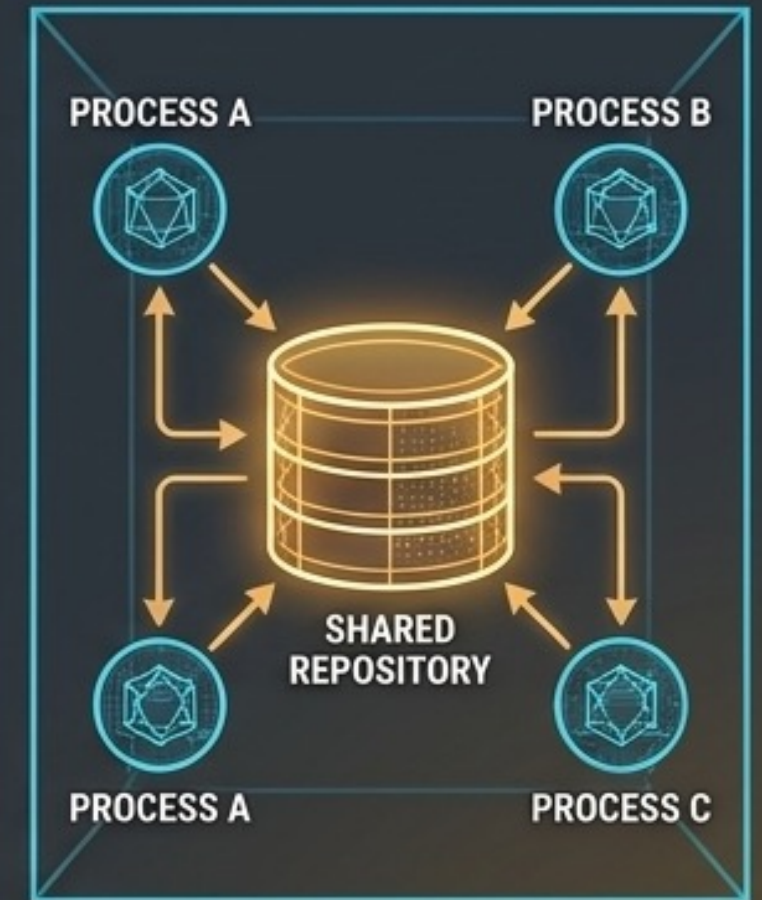
2. Object-Based

Encapsulated objects
invoke methods across the
network via proxies.



3. Event-Based

Processes communicate
strictly via event propagation
(Publish/Subscribe).



4. Data-Centered

Processes communicate
by reading/writing to a
persistent shared repository.

System Architectures: Centralized vs. Decentralized

Centralized

Traditional multi-tiered Client-Server setups (thin vs. fat clients).

Overlay Topography

Structured Overlay (DHTs)

Decentralized Peer-to-Peer (P2P)

Structured Overlays:
Use Distributed Hash Tables (DHTs) for deterministic routing.

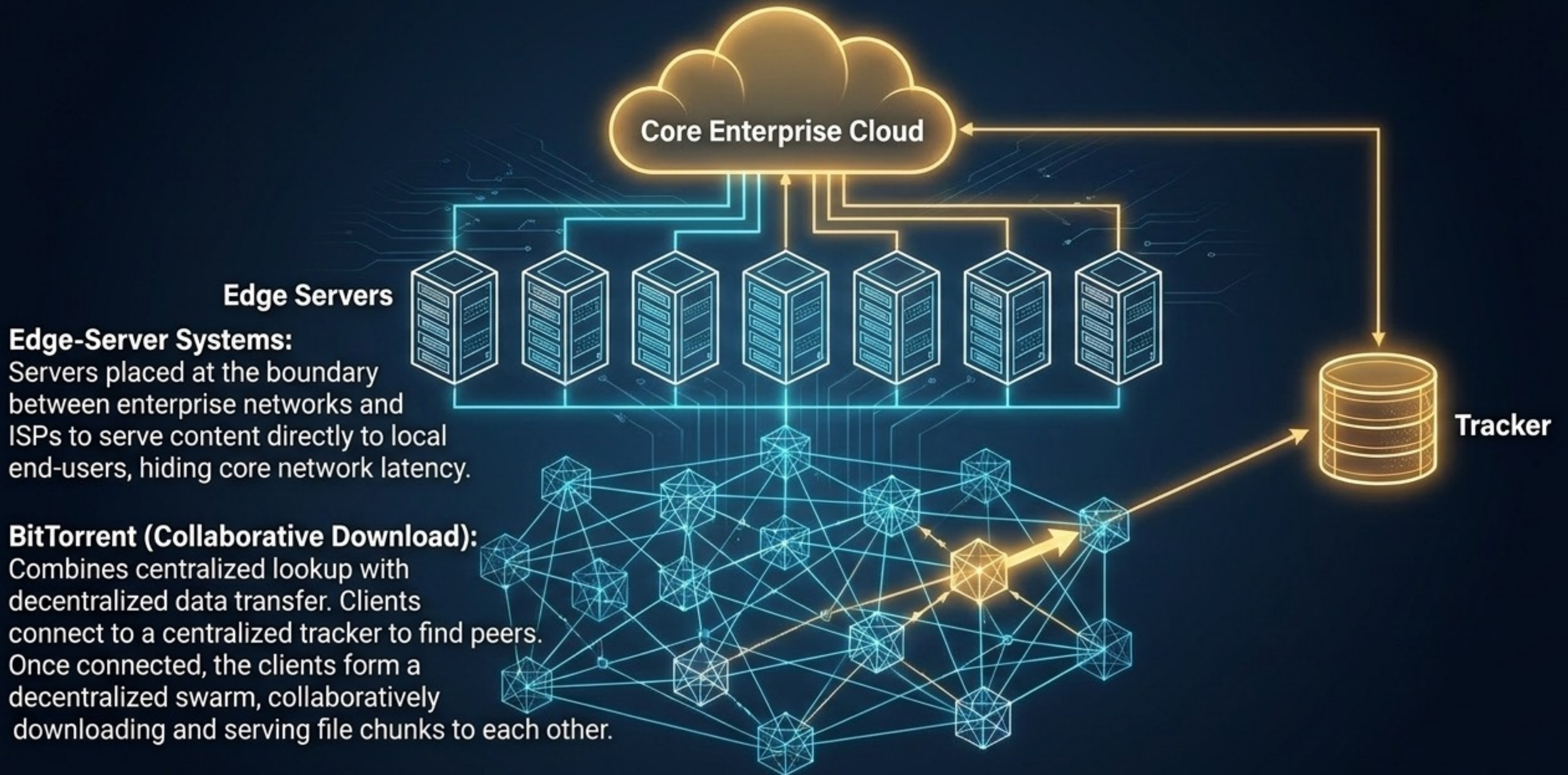
- Chord rings: 1D circular ID spaces.
- CAN: d-dimensional Cartesian spaces.

Unstructured Overlays

Randomized connections relying on gossip protocols, partial views, and search algorithms.

Chaotic Physical Substrate

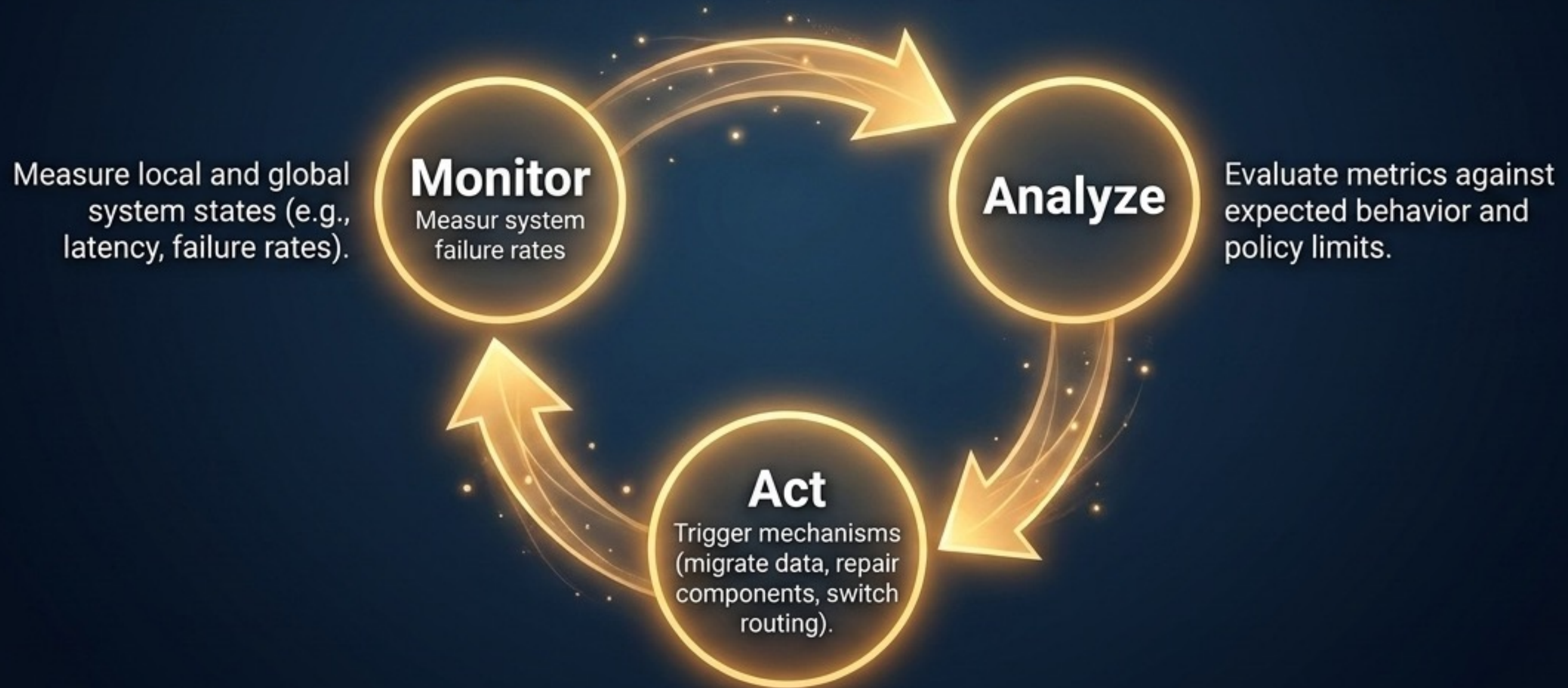
System Architectures: Hybrid Solutions



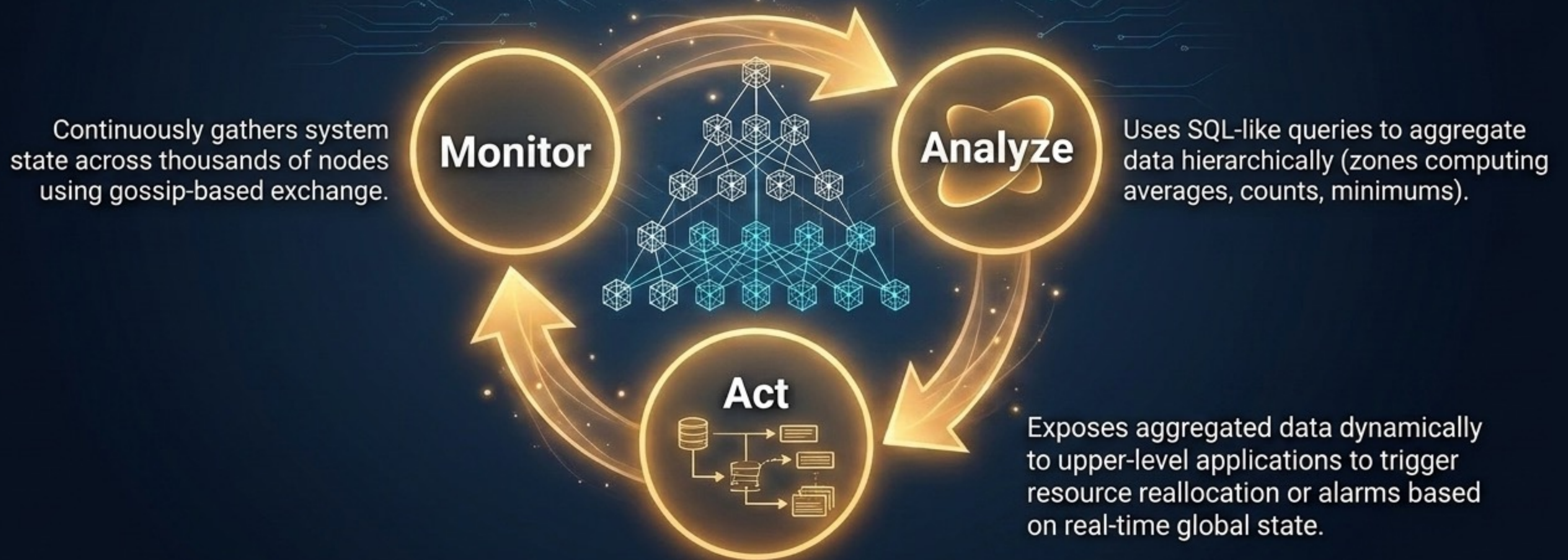
The Autonomous Magician: Self-Management

The Problem: Geographic and administrative scale outpaces human sysadmins' ability to configure policies manually.

The Solution: Adaptive software via Feedback Control Loops.



Self-Management Example 1: Systems Monitoring (Astrolabe)

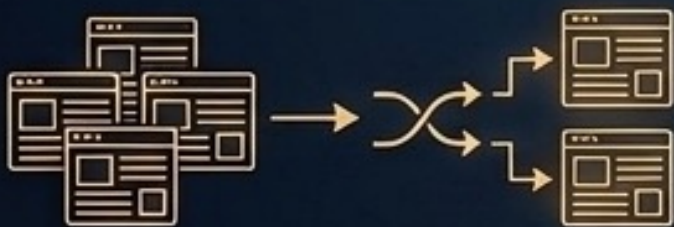


Self-Management Examples 2 & 3

Globule (Replicated Strategy Differentiation)

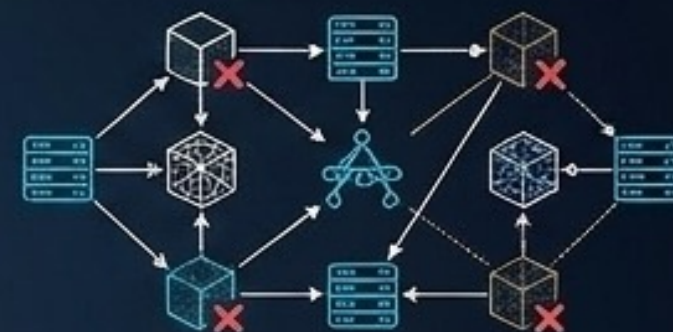


Monitor/Analyze: Runs trace-driven what-if simulations evaluating the cost of different replication policies (master/slave, invalidation, caching) using finite optimal trace lengths.

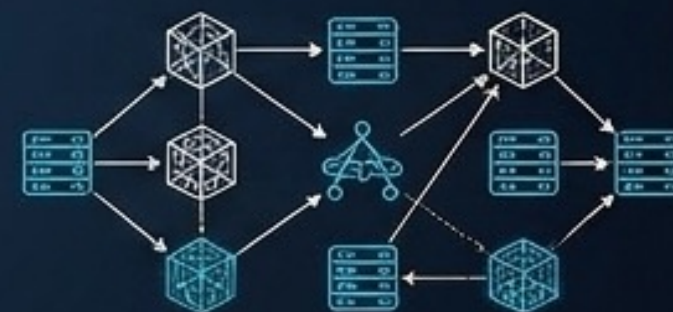


Act: Dynamically switches the replication policy per Web page to minimize access costs.

Jade (Automatic Component Repair)



Monitor/Analyze: Detects node or network failures in the system topology.



Act: Automatically adds new nodes to the cluster and reconfigures the cluster and reconfigures the software architecture without halting the system.

The Apprentice's Test: Concept Review



Review Question 1: Can you map a real-world scenario to all 7 ISO Transparencies? Where do they inherently conflict?



Review Question 2: What are the 4 fundamental rules of decentralized algorithms? Why does a central clock violate them?



Review Question 3: How does the separation of Policy and Mechanism directly support the goal of Openness?

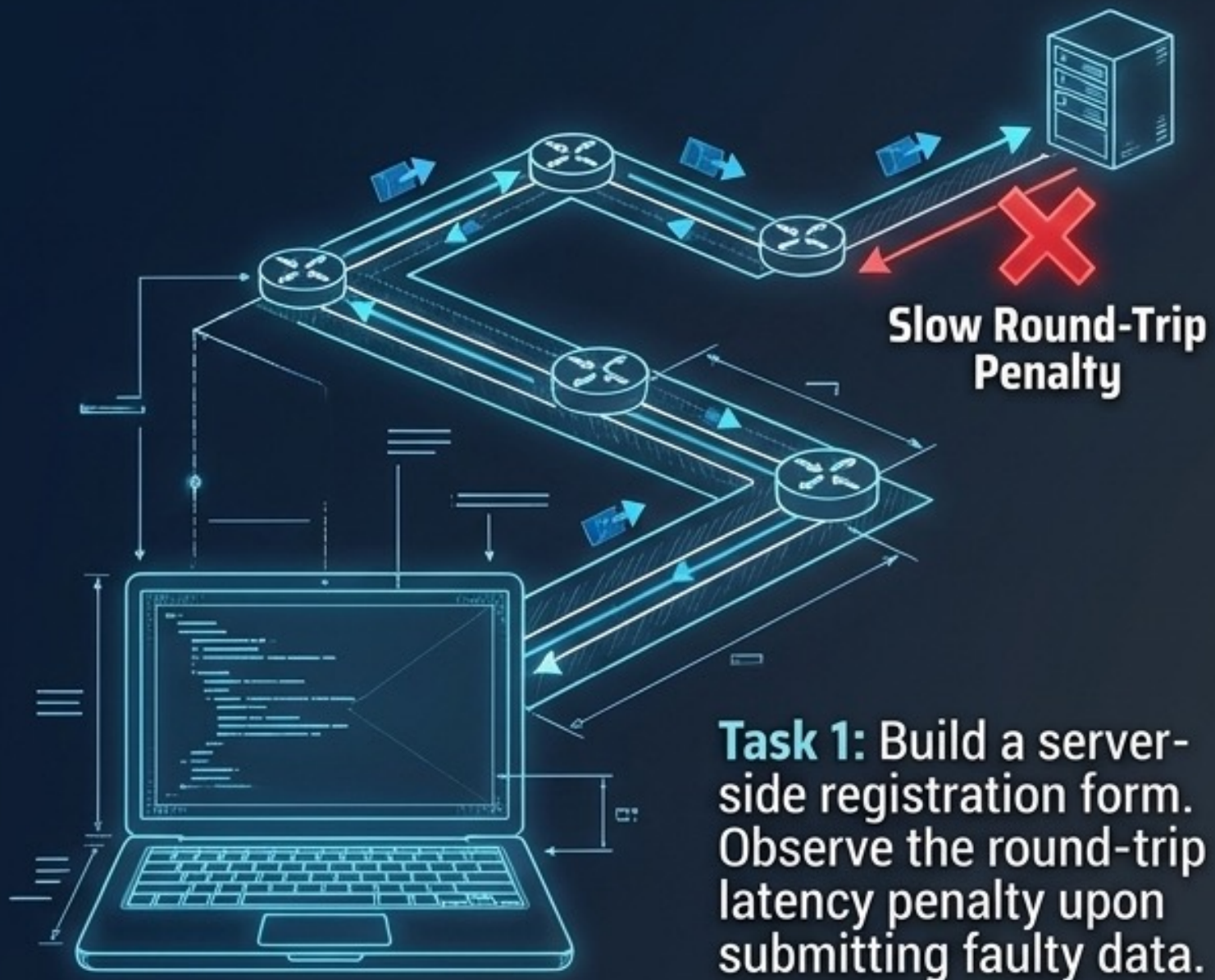


Review Question 4: In an autonomic feedback loop, how does Astrolabe's hierarchical aggregation feed the Analyze phase?

Practical Lab: Architecting Latency Hiding

The Mission: Design and implement a client-server split layout to intentionally hide WAN latency, sustaining The Illusion of One.

Traditional Network (High Latency)



Latency Hiding (Local Validation)

Task 2: Ship code to the client (JavaScript) to validate data formatting locally before transmission.



Task 3: Measure and map the performance delta. Document which of Peter Deutsch's 8 False Assumptions your local validation mitigates.